

Практика: Множество на основе хеш-таблицы

Юрий Литвинов
y.litvinov@spbu.ru

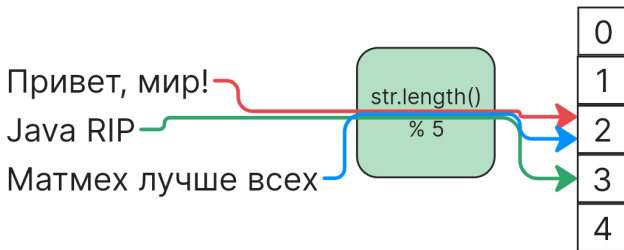
26.09.2026

Напоминание про хеш-таблицы

- ▶ Вероятностная структура данных, основа для реализации множеств и словарей
- ▶ Позволяет достичь «почти» константного времени выполнения базовых операций (вставки, удаления, проверки на принадлежность/поиска) при довольно небольших накладных расходах
 - ▶ ведёт себя как массив, где индексами могут быть значения произвольного типа
- ▶ Хранят значения «перемешанными»
- ▶ Реализуются на массивах (т.н. «открытая адресация») и на списках
 - ▶ на массивах работает побыстрее, но в образовательном смысле на списках нам интереснее

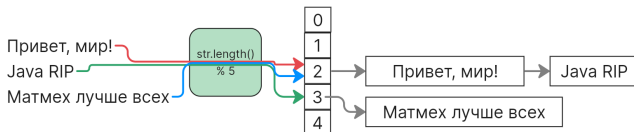
Хеш-функции

- ▶ Некоторая функция, отображающая большое (потенциально бесконечное) множество ключей в конечное (и маленькое) множество хеш-значений
- ▶ Чем «случайнее» она это делает, тем лучше
- ▶ Зачем: факторизуем множество ключей по классам эквивалентности, образованным ключами с равными хеш-значениями, будем хранить в массиве фактор-множества
 - ▶ чем лучше хеш-функция, тем равномернее значения будут раскладываться по хеш-таблице



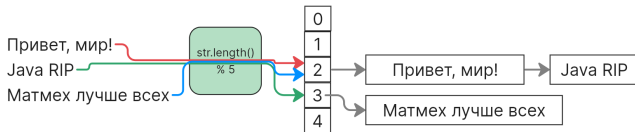
Хеш-таблицы на списках

- ▶ Заведём массив с числом элементов, примерно равным ожидаемому размеру множества
- ▶ Сделаем хеш-функцию, множество значений которой — индексы массива
 - ▶ проще всего — взять хеш-функцию, которая просто считает неотрицательное целое, и брать остаток от деления на длину массива
- ▶ В каждой ячейке массива — список из значений
- ▶ Если у двух элементов хеш-значения совпадают, просто кладём их в список подряд



Базовые операции

- ▶ Добавление: считаем хеш-функцию, находим индекс в массиве, добавляем в голову списка
 - ▶ $O(1)$
- ▶ Удаление: считаем хеш-функцию, находим индекс в массиве, бежим по списку, сравниваем, если нашли — удаляем из списка
 - ▶ $O(\text{длина списка})$
- ▶ Поиск: считаем хеш-функцию, находим индекс в массиве, бежим по списку, сравниваем, если не нашли — элемента нет
 - ▶ $O(\text{длина списка})$



Тонкости

- ▶ Если хеш-функция хорошая, длина списка обычно равна количеству элементов множества, делённому на количество ячеек в массиве
 - ▶ это называется «коэффициент заполнения» (load factor)
 - ▶ в идеале для хеш-таблиц на списках коэффициент заполнения должен быть равен 1
 - ▶ тогда базовые операции имеют константную трудоёмкость
 - ▶ слишком низкий коэффициент заполнения — лишние расходы памяти
- ▶ Количество хранимых данных, как правило, заранее неизвестно, поэтому хеш-таблица динамически меняет размер
 - ▶ при каждом добавлении проверяем коэффициент заполнения, если он больше 1, увеличиваем массив вдвое
 - ▶ размер массива участвует в хеш-функции, так что все элементы надо переложить
 - ▶ иначе их потом не найти будет

Выбор хеш-функции

- ▶ Пример достаточно хорошей хеш-функции для строки:
 $a[0] * p^n + a[1] * p^{n-1} + \dots + a[n]$, особенно если p — простое (Rolling hash)
 - ▶ Считается в цикле «умножили предыдущее значение на p и прибавили следующую букву»
- ▶ Для чисел часто используется тождественная функция
- ▶ Есть `Object.hashCode()`, переопределяемый для конкретных классов
 - ▶ как раз тождественная функция для чисел
 - ▶ по умолчанию значение указателя для ссылочных типов
 - ▶ возвращает произвольное целое (может быть отрицательным), его надо самим приводить в нужный диапазон
- ▶ Бывают ещё криптографические хеш-функции (MD5, SHA-1, SHA-3, ...), они долго считаются
- ▶ Бывает идеальный хеш — инъективная функция (т.е. не допускает коллизий)

Задача

- ▶ Поделиться на команды по 2-3 человека
- ▶ Реализовать класс `StringSet` — реализацию АТД «Множество» для строк на хеш-таблицах
- ▶ Должны поддерживаться операции:
 - ▶ **void** `put`(String element) — добавить элемент в множество
 - ▶ **bool** `remove`(String element) — удалить элемент из множества
 - ▶ возвращает `true`, если операция завершилась успешно
 - ▶ **boolean** `contains`(String element) — `true`, если данная строка принадлежит множеству
 - ▶ **int** `size`() — количество элементов множества
 - ▶ **void** `clear`() — очистить множество

Задача, ещё требования

- ▶ Для реализации потребуется список, он должен быть реализован как отдельный класс
 - ▶ это должен быть связный список, на ссылках
 - ▶ достаточно только тех операций, которые нужны в данной задаче
- ▶ Нужен код в `main`, который проверяет, что всё работает
 - ▶ но не увлекайтесь, на следующем занятии будем писать тесты
- ▶ Должно быть поддержано динамическое изменение размера хеш-таблицы